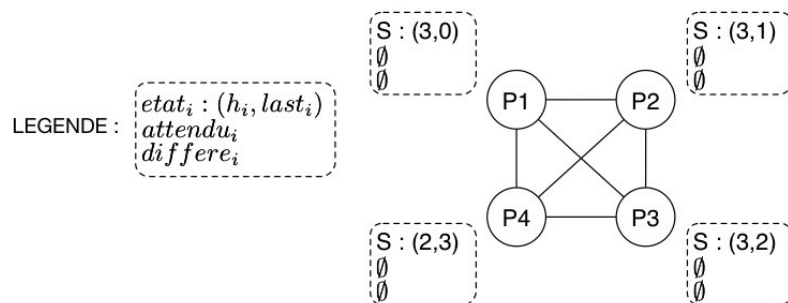


Exercice 1. Algorithme basé sur les permissions de Ricart-Agrawala

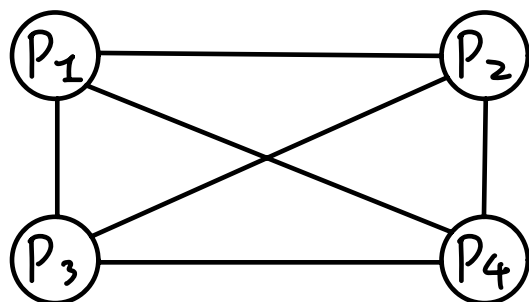
On considère l'algorithme d'exclusion mutuelle de Ricart-Agrawala et la configuration initiale C_0 suivante :



On dit que deux sites P et P' demandent "simultanément" à entrer en SC si, au cours d'une configuration, les sites P et P' sont tous les deux dans l'état de demande de section critique.

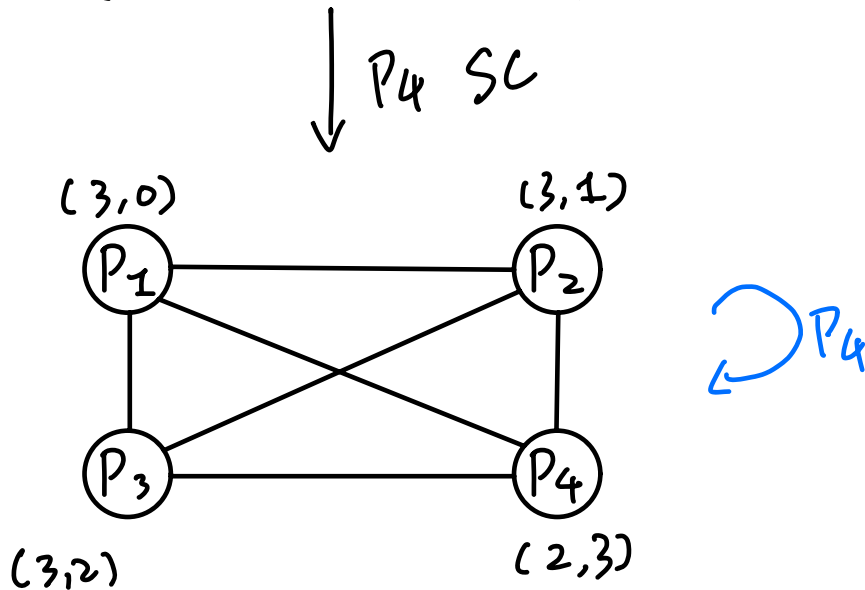
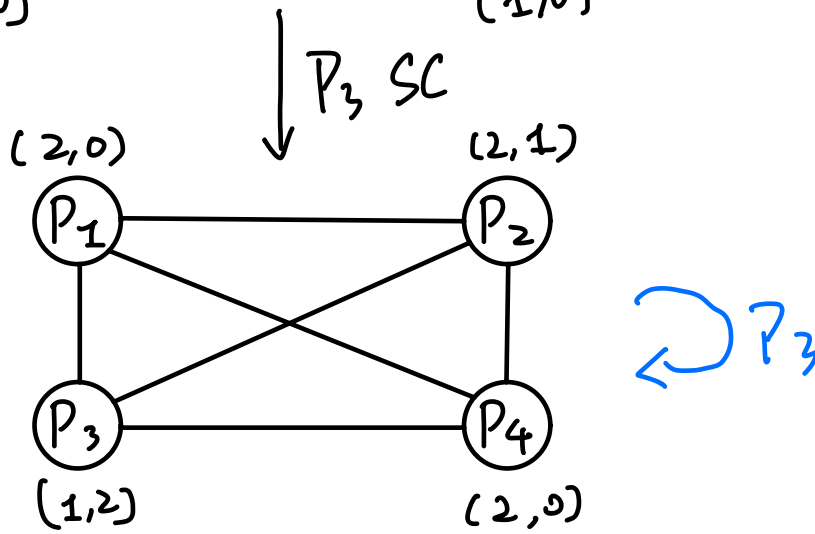
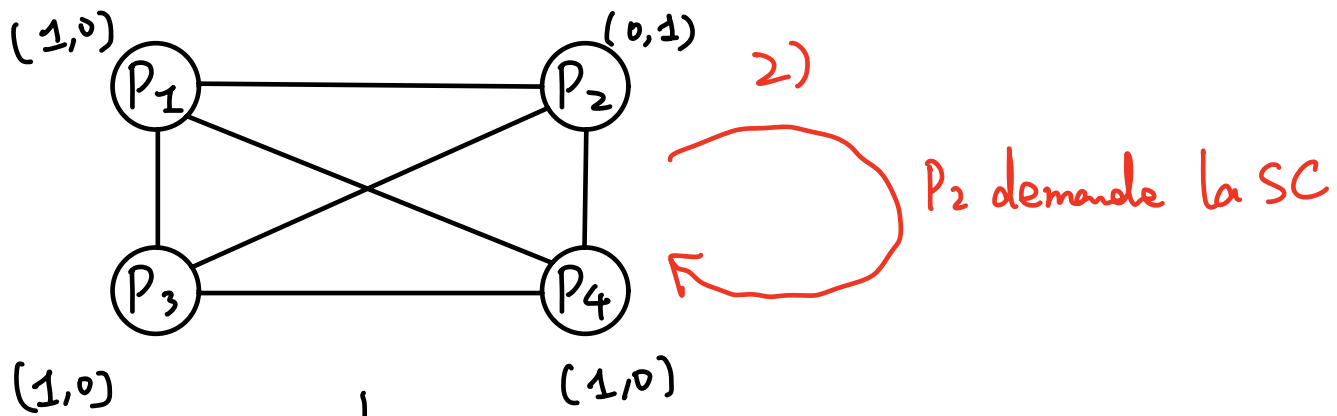
- Donner les grandes lignes d'une exécution qui a pu aboutir à la configuration C_0 et dans laquelle il n'y a pas de demandes simultanées et chaque site fait au plus une demande de SC.
- Donner les grandes lignes d'une exécution qui a pu aboutir à la configuration C_0 et dans laquelle il n'y a pas de demandes simultanées et P_2 fait deux demandes de SC.
- En déduire combien de demandes de SC ont fait P_1 , P_2 , P_3 et P_4 pour arriver à cette configuration ?
- A partir de C_0 , on suppose que les sites P_1 et P_3 demandent en même temps à entrer en section critique.
 - Dire qui de P_1 ou de P_3 entre en premier en section critique. Justifier.
 - Donner la valeur des variables h_i et last_i pour tous les sites, après que P_1 et P_3 soient sortis de section critique.
 - Indiquer combien de messages ont été échangés dans cette exécution.
- A partir de C_0 , comment faire pour qu'un site demande son entrée en section critique, mais voit sa demande différée à cause de son estampille.

$$\begin{array}{ll} 1) & (h_1, \text{last}_1) = (0, 0) \\ & \text{attendu}_1 = \emptyset \\ & \text{differe}_1 = \emptyset \end{array} \quad \begin{array}{ll} & (0, 0) \\ & \text{attendu}_2 = \emptyset \\ & \text{differe}_2 = \emptyset \end{array}$$

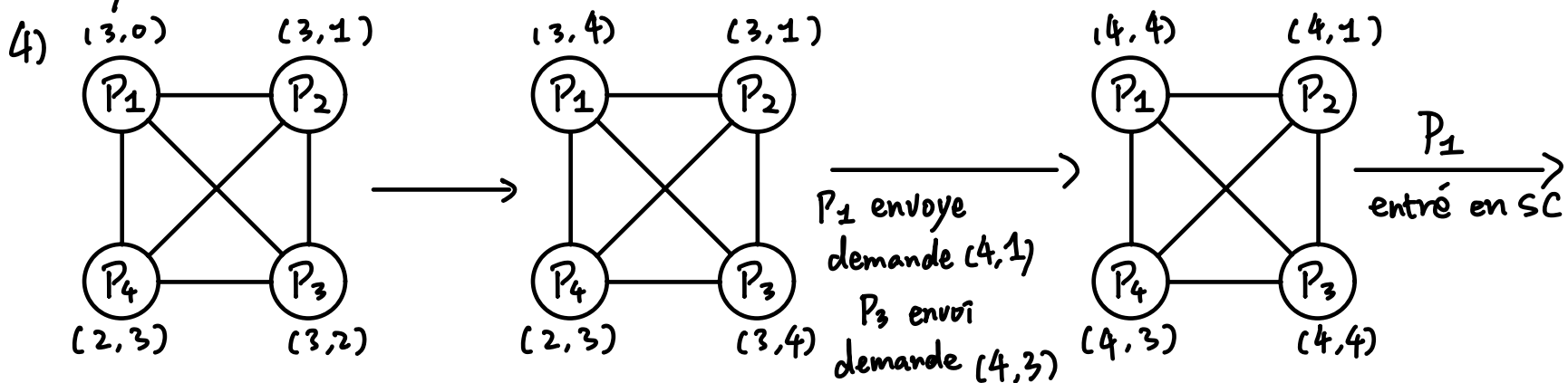


$$\begin{array}{ll} (0, 0) & (0, 0) \\ \text{attendu}_3 = \emptyset & \text{attendu}_4 = \emptyset \\ \text{differe}_3 = \emptyset & \text{differe}_4 = \emptyset \end{array}$$

P_2 demande la SC $\downarrow$ On suppose les autres nœuds lui envoient la permission.



3) P_1 fait 0 demande, $P_2, P_3, P_4 \infty$: demande consécutive (pour chaque nœud)



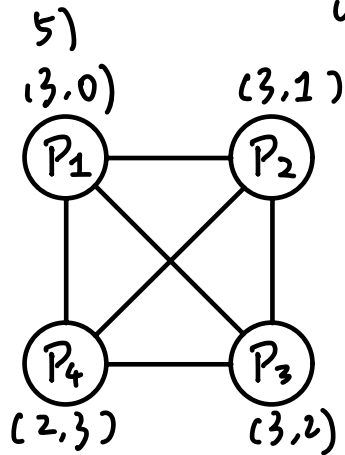
a) P_1 rentrera en premier

c) P_1 envoi 3 Dem; P_2, P_3, P_4 envoi Perm à P_1

priorité ₁ = Vrai	$(4,1) < (4,3)$
priorité ₃ = Faux	$(4,3) \nless (4,1)$

P_3 envoie 3 Dem, P_2, P_4 envoi Perm à P_3 ; P_1 envoi Perm à P_3

→ 12 messages $O(n \times \# \text{ demande entrées en SC})$



$P_4: \text{dem}(3,4)$: priorité : $(3,4) < (4,2)$

$P_2: \text{dem}(4,2)$: : $(4,2) < (3,4)$

↳ perm à P_4

P_2 voit sa demande refusée à cause de son horloge

Exercice 2. Algorithme basé sur les permissions de Carvalho-Roucairol

L'objectif de cet exercice est d'écrire une version améliorée de l'algorithme de Ricart-Agrawala, version qui améliore la complexité en messages. Le principe de ce nouvel algorithme est donné ci-dessous.

On considère que, pour chaque paire de sites (i, j) , il existe un unique jeton nommé $Perm(i, j)$. Un site i ne peut entrer en SC que s'il possède pour tout j le jeton $Perm(i, j)$. Lorsqu'un site veut entrer en SC, il fait donc des requêtes pour tous les jetons qui lui manquent aux sites concernés, et uniquement pour ceux-ci. Lorsqu'un site i reçoit d'un site j la demande de permission $Perm(i, j)$, il l'envoie uniquement s'il calcule qu'il n'est pas prioritaire ; sinon, il enverra le jeton après qu'il soit entré en SC. Le calcul des priorités se fait à l'aide d'estampillages de manière analogue à l'algorithme précédent.

En pratique, on n'utilise pas explicitement ces jetons ; si le site i n'a pas $Perm(i, j)$, on traduit cela sur le site i par $j \in \text{attendu}_i$.

Il y a une partie délicate dans cette version : lorsqu'un site i est en train d'attendre les permissions qu'il a demandées et qu'il reçoit alors d'un site j une demande prioritaire de permission, alors i doit non seulement envoyer sa permission à j , mais il doit en plus faire une demande de permission à j (envoi de deux messages : Perm puis Dem à j). En effet, au moment où i envoie sa permission à j , il détient la permission de j , il n'a donc aucune demande de permission à j en cours. Le site i doit donc indiquer à j de lui redonner la permission une fois sorti de SC, afin qu'il puisse lui-même entrer en SC après j .

L'algorithme de Ricart-Agrawala est donné ci-dessous. Insérer les modifications nécessaires pour mettre en place la version améliorée.

Variables et initialisations

...

Algorithme

Sur demande d'entrée en section critique →

```
etat_i ← E
last_i ← h_i + 1
attendu_i ← tous les sites sauf i
for all j ∈ attendu_i do
    Envoyer Dem(last_i, i) à j
```

Sur réception de Perm de j →

```
attendu_i ← attendu_i \ {j}
if attendu_i = ∅ then
    etat_i ← SC
```

Sur réception de Dem(h', j) de j →

```
h_i ← max(h_i, h')
priorite_i ← (etat_i = SC) ∨
               [(etat_i = E) ∧ (last_i, i) < (h', j)]
if priorite_i then
    differe_i ← differe_i ∪ {j}
else
    Envoyer Perm à j
    ...
```

Sur sortie de section critique →

```
etat_i ← S
for all j ∈ differe_i do
    Envoyer Perm à j
    ...
differe_i ← ∅
```

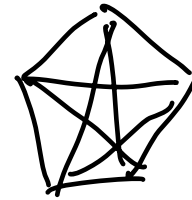
Exercice 3. Algorithme basé sur la circulation d'un jeton de Ricart-Agrawala

- On considère un graphe complet de 5 sites : P_1, P_2, P_3, P_4 et P_5 .
 - Donner l'exécution de l'algorithme dans le scénario suivant : il y a une seule demande d'entrée en SC et celle-ci est faite par le site P_1 .
 - Donner un scénario dans lequel le site P_4 est le deuxième à entrer en SC, alors que c'est le premier à en faire la demande.
- Quel est le temps d'attente maximal entre 2 entrées successives en SC d'un même site ? Quel est le temps d'attente minimal ?
- Illustrer au travers d'une exécution la raison pour laquelle un site parcourt son tableau et celui du jeton de l'indice $i + 1$ jusqu'à n , puis de l'indice 1 à $i - 1$ et non de l'indice 1 à n , privé de l'indice i .

1)

	nbDem	Jeton	Valeur Jeton
P_1	00000 → 10000	F V	00000 → 10000 (quand P_1 sort de la SC)
P_2	00000 → 10000	F	
P_3	00000 → 10000	F	
P_4	00000 → 10000	F	
P_5	00000 → 10000	V	00000 → 00000

V : vrai F : faux



Initialement :

P_1 : exécute entrée en SC + envoi Dem ()

P_5 : exécute réception Dem + sortie de SC

⋮

for Jeton[0] < nbDem₅[0]
0 < 1

⇒ Jeton envoyé à P_1

P_1 entre en SC, Il en sort et met à jour le jeton.

b) P_1 a le jeton. / tableau nbDem₁ : 10000

P_1 demande la SC → il l'obtient



• P_4 demande la SC

↳ tous ses message Dem() sont reçus

10010

↓
10110

• P_3 idem

↙ envoyer Jeton à 3

• P_1 sort de SC

Jeton = 10000

2) temps minimum : 0 si le jeton est sur le noeud

temps maximum pour j : si le jeton est sur le noeud $j+1$ et que tous les noeuds demandent la SC : (n noeuds) $\Rightarrow O(n)$ temps.

3) Jeton sur P_1

- P_1 demande la SC et y entre
- P_2 demande la SC
- P_3 _____
- P_1 sort de SC $\rightarrow$ jeton sur P_3
- P_1 demande la SC
- P_3 sort de SC $\rightarrow$ envoyer Jeton à P_2 .

Soit Δ le degré maximal d'un graphe.

3. La complexité de l'algorithme de Ricart Agrawala est asymptotiquement:

- ☒ (a) $\Delta * (\text{nombre de demandes})$. (b) $n * (\text{nombre de demandes})$. (c) $\Delta * n$. (d) $\Delta * m$.

4. Il est possible d'écrire un algorithme d'élection de leader fonctionnant sur n'importe quel type de réseau.

- (a) Vrai. ☒ (b) Faux.

5. L'algorithme de Ricart Agrawala fonctionne si les canaux de communications ne sont pas FIFO:

- ☒ (a) Vrai. (b) Faux.

6. Dans notre modèle algorithmique, les variables de connaissance sont accédées par les noeuds:

- (a) En écriture. ☒ (b) En lecture. (c) En lecture et en écriture.

7. Dans notre modèle algorithmique, lorsqu'un noeud effectue un STOP-GLOBAL, cela signifie:

- (a) il a terminé d'exécuter le code mais d'autres noeuds non.
(b) tous les noeuds ont terminé d'exécuter le code mais des messages peuvent être en transit.
☒ (c) tous les noeuds ont terminé d'exécuter le code et il n'y a pas de messages en transit.